AI-Powered Chatbot for University Student Support

HCLTech Career Shaper Programme: III Year Industry Project (Project No. 30)

Submitted by: Raj Porwal (RaidX), B.Tech CSE, ABES Engineering College, Ghaziabad | Academic Year 2025-26 | Domain: Digital-Data-AI / AIML | Mentor: [Mentor name]

> **How to finish this document:** every item in [square brackets] is a placeholder for a number, screenshot or name that only your own run can supply. Replace them with your real results before submission. Do not submit invented figures.

Abstract

University helpdesks answer the same questions about fees, exams, hostels, scholarships and admissions thousands of times a term, and students often wait hours for a reply. This project builds an NLP chatbot that classifies a student's intent, retrieves the best answer from a curated FAQ knowledge base, and escalates to a human agent when the bot is unsure, the student is frustrated, or the topic is sensitive. The system combines a TF-IDF and sentence-embedding intent classifier, a semantic FAQ retriever, a sentiment analyser, and a rule-based escalation engine, exposed through a REST API and a Streamlit chat interface. It was trained and validated on a chat-log dataset of [N] sessions across [K] intents. On a held-out, session-grouped test set the best intent model reached a macro F1 of [value], FAQ retrieval reached Top-3 accuracy of [value], and the end-to-end system achieved a containment rate of [value] with a median latency of [value] seconds. Privacy by design is central: no personally identifiable information (PII) is stored beyond the session, users can opt out, and every escalation is clearly announced.

Keywords: intent classification, FAQ retrieval, sentiment analysis, human escalation, university helpdesk, sentence embeddings, RAG

1. Introduction

1.1 Problem Statement

Student support teams handle large volumes of repetitive queries. Peaks around admissions, fee deadlines and examinations overload staff, answers are inconsistent between agents, and out-of-hours students get no help at all. At the same time, a small

share of queries (grievances, harassment, mental-health concerns, payment disputes) need a human immediately. A useful chatbot must therefore do two things well: answer routine questions accurately and know when not to answer.

1.2 Objectives

1. Build an intent classifier that maps a student message to one of [K] helpdesk intents, including an out-of-scope class.
2. Build an FAQ retrieval module that returns a grounded answer only when it is confident.
3. Add sentiment analysis to detect frustration and urgency.
4. Implement an escalation policy that hands the conversation to a human agent with full context, and make that handoff visible to the student.
5. Validate every component against measurable targets and document the findings.
6. Comply with the programme's ethical requirements on PII, opt-out and transparent escalation.

1.3 Scope

In scope: English and lightly code-mixed (Hinglish) text queries, a single university domain, a web chat interface, and a ticket store for escalated chats. Out of scope: voice, live integration with the university ERP or LMS, and payment processing.

1.4 Report Organisation

Section 2 reviews tools and related work. Section 3 describes the dataset and preprocessing. Section 4 presents the system design and Section 5 the implementation. Section 6 reports results and error analysis. Sections 7 to 9 cover ethics, limitations and future work, and Section 10 concludes.

2. Literature and Tools Review

2.1 Approaches to Helpdesk Chatbots

Rule-based and menu-driven bots are predictable but brittle: any phrasing outside the script fails. Retrieval-based bots match a query to a stored answer and are easy to audit, which suits FAQ-style support. Generative bots produce fluent replies but can hallucinate policies, which is unacceptable for fees, deadlines and rules. A hybrid design, in which a classifier and a retriever choose the answer and a generator (if used at all) only rephrases retrieved text, offers the best balance of coverage and safety. This project follows the hybrid, retrieval-first approach.

2.2 Intent Classification

Classical pipelines use bag-of-words or TF-IDF features with Logistic Regression or linear SVMs. They train in seconds, work with small data and are a strong baseline. Transformer encoders such as BERT and DistilBERT, and sentence-embedding models such as all-MiniLM-L6-v2, capture meaning rather than surface words and handle paraphrase better, at higher compute cost. The report compares a TF-IDF baseline against an embedding-based classifier.

2.3 FAQ Retrieval

Semantic search encodes questions and FAQ entries into dense vectors and ranks them by cosine similarity. A similarity threshold decides between answering and falling back. This is the retrieval half of retrieval-augmented generation (RAG), without the generation step unless needed.

2.4 Sentiment Analysis

VADER is a lexicon-and-rule model that needs no training and runs instantly, making it a good baseline for short informal text. Transformer models such as cardiffnlp/twitter-roberta-base-sentiment are more accurate on social-media-style language. Sentiment is used here as an escalation signal, not as a standalone product.

2.5 Tools Used

Layer	Tool	Reason
Language	Python 3	ML ecosystem, programme requirement
Classical ML	scikit-learn	TF-IDF, Logistic Regression, metrics
Embeddings	sentence-transformers	Pretrained semantic encoders
Text utilities	NLTK	Tokenisation, stopwords, VADER
Retrieval	FAISS or NumPy cosine search	Lightweight, no server needed
Backend	Flask (REST API)	Listed in the programme toolset
Storage	SQLite	Ticket store and aggregated analytics
Frontend	Streamlit	Fast, demo-ready chat UI

3. Dataset Description and Preprocessing

3.1 Source and Schema

The project uses the programme-provided chat-log file `dataset.csv` ([N] rows, [S] unique sessions). Each row records one student interaction with a helpdesk bot. The columns used are listed below and described fully in Appendix A (`data_dictionary.csv`).

Column	Type	Role in this project
session_id	string	Grouping key for train/test split and repeat-question detection
user_query_type	categorical	Coarse query category, used for analysis
paraphrased_query	text	Model input (X)
intent	categorical	Target label (y) for the intent classifier
confidence_score	float	Confidence logged by the original system; analysed but not used as a feature
escalated	Y/N	Ground truth for the escalation evaluation and for containment rate
user_rating	integer	Satisfaction metric and weak sentiment signal
timestamp	datetime	Temporal analysis and demand patterns

Note: if your copy of the file also contains `bot_response`, it is used as the source for FAQ answers (see Section 5.3).

3.2 Exploratory Data Analysis

The EDA notebook (`01_eda.ipynb`) contains the following visualisations. Insert the figures and one sentence of insight under each.

1. **Intent distribution** (bar chart): [Finding, for example class imbalance and the largest and smallest intents].
2. **Escalation rate per intent** (bar chart): [Finding, for example which intents are escalated most].
3. **Rating distribution** (histogram): [Finding].
4. **Logged confidence score distribution, split by escalated vs not** (histogram): [Finding, whether low confidence predicts escalation].

5. **Queries per hour or weekday** (line chart from `timestamp`): [Finding, peak periods].
6. **Query length distribution** (histogram): [Finding].

3.3 Cleaning and Preprocessing

Step	Action	Rows affected
Missing values	Dropped rows with empty `paraphrased_query` or `intent`	[n]
Duplicates	Removed exact duplicates of (query, intent)	[n]
Label hygiene	Merged inconsistent intent names (case, spacing, synonyms)	[n]
Type fixes	Parsed `timestamp`, mapped `escalated` Y/N to 1/0	all
Text normalisation	Lower-casing, whitespace trimming, URL and number masking; no aggressive stopword removal (it hurts short queries)	all
PII scrubbing	Regex masking of emails, phone numbers and roll-number patterns before any text is stored	[n]

3.4 Train, Validation and Test Split

Paraphrases of the same underlying query can appear in different rows, so a random row split would leak near-duplicates into the test set and inflate scores. The data was therefore split **by `session_id`** using grouped splitting: 70% train, 10% validation, 20% test. The validation set is used to tune thresholds; the test set is touched once for the final numbers.

3.5 Sentiment Labels

The dataset has no sentiment column. Two sources are combined: (a) VADER or RoBERTa predictions on `paraphrased_query`, and (b) weak labels from `user_rating` (1 to 2 negative, 3 neutral, 4 to 5 positive). A sample of [100] messages was hand-labelled to measure how well each source agrees with a human judgement.

4. System Design

4.1 Architecture

```
Student message
      |
      v
PII scrub + normalise
      |
      +--> Intent classifier --> intent + confidence
      +--> Sentiment analyser --> label + score
      |
      v
FAQ retriever (top-k cosine similarity, filtered by intent)
      |
      v
Escalation engine
      |-- no trigger --> grounded FAQ answer (+ feedback buttons)
      |-- trigger -----> create ticket, notify student, hand off to human
agent
```

4.2 Components

Preprocessor. Masks PII, normalises text and detects the explicit phrase "talk to a human" (and Hinglish equivalents such as "kisi insaan se baat karao").

Intent classifier. Predicts one of [K] intents including `out_of_scope`, and returns a calibrated probability used as confidence.

Sentiment analyser. Returns negative, neutral or positive plus a score. Strongly negative messages are treated as a frustration signal.

FAQ retriever. Searches the embedded FAQ base, restricted first to the predicted intent and then, if nothing clears the threshold, to the whole base. Returns the best answer and its similarity score.

Escalation engine. Applies the policy in Section 4.3.

Session manager. Holds conversation state in memory only, tracks repeated questions, and discards everything when the session ends.

Ticket store (SQLite). Stores only what an agent needs: a ticket ID, the masked transcript, the detected intent, the sentiment and the timestamp.

4.3 Escalation Policy

The conversation is escalated to a human if **any** of the following holds:

#	Trigger	Default threshold (tuned on validation set)
1	Strongly negative sentiment	compound score at or below [-0.5]
2	Low intent confidence	probability below [0.60]
3	Weak retrieval match	cosine similarity below [0.45]
4	Explicit request for a human	phrase match
5	Same question repeated	2 or more times in a session
6	Sensitive intent	harassment, mental health, grievance, payment dispute

Thresholds were chosen on the validation set to balance two costs: unnecessary escalations (agent workload) against missed escalations (student frustration). Triggers 4 and 6 are always enforced regardless of the other scores.

4.4 Data Flow and Privacy

Queries are scrubbed before they reach any log. Conversation state lives only in session memory. A ticket is written only on escalation, and aggregated analytics (counts per intent, ratings) contain no message text. An opt-out control lets a student end data capture and go straight to a human at any time.

4.5 API Design

Endpoint	Method	Purpose
/chat	POST	Takes a message and session ID, returns the answer, intent, confidence and an `escalated` flag
/escalate	POST	Creates a ticket for the current session and returns the ticket ID
/feedback	POST	Records a 1 to 5 rating for a bot reply
/tickets	GET	Admin view of escalated tickets
/health	GET	Liveness check

5. Implementation

5.1 Environment and Repository Structure

```
chatbot/
  data/            dataset.csv, data_dictionary.csv, faq.csv
  notebooks/       01_eda.ipynb, 02_intent_models.ipynb, 03_evaluation.ipynb
  src/
    preprocess.py  pii.py  intent.py  sentiment.py
    retriever.py   escalation.py  session.py
  api/app.py       Flask REST API
  ui/streamlit_app.py
  tests/
  models/
  README.md
```

5.2 Intent Classification

Two models were trained on the same grouped split.

1. **Baseline:** TF-IDF (unigrams and bigrams, sublinear term frequency) with Logistic Regression and balanced class weights.
2. **Embedding model:** sentence embeddings from all-MiniLM-L6-v2 (or paraphrase-multilingual-MiniLM-L12-v2 for Hinglish) fed to Logistic Regression.
3. **Optional:** fine-tuned DistilBERT as a third comparison row.

The predicted probability of the winning class is the system's own confidence score. The logged `confidence_score` column is deliberately not used as a feature because it leaks information from the system that produced the logs.

5.3 FAQ Knowledge Base and Retrieval

A knowledge base of [40] question-answer pairs was written from the university's published information (fees, examinations, admissions, hostel, scholarships, placements, timetable, certificates), each tagged with an intent. Questions are embedded once and stored in a FAISS index (or a NumPy matrix). At runtime the query embedding is compared with the entries for the predicted intent, and the top-3 are returned with their scores. If the best score is below the retrieval threshold the bot says it is not sure and triggers the escalation check instead of guessing. Answers are returned verbatim from the knowledge base so the bot cannot invent policy.

5.4 Sentiment Analysis

VADER (NLTK) is the baseline. A pretrained RoBERTa sentiment model is evaluated against it on the hand-labelled sample. The better model per macro F1 is deployed; the other is documented as a comparison.

5.5 Escalation Engine

The engine is a small, transparent function that receives intent, confidence, sentiment score, retrieval score and session history, and returns `(escalate: bool, reason: str)`. Returning a reason makes every handoff explainable to the student and to the agent. Because it is rule-based, it can be audited line by line, which matters for a student-facing service. A learned escalation classifier trained on the `escalated` column was compared with the rules; see Section 6.4.

5.6 Backend and Interface

The Flask API wraps the pipeline behind the endpoints in Section 4.5. The Streamlit interface shows the chat, a visible banner reading "Connecting you to a human agent" on escalation, thumbs-up and thumbs-down feedback buttons, and an opt-out button. A second Streamlit page gives staff a ticket queue and intent-distribution charts.

5.7 Testing

Unit tests cover the PII scrubber, each escalation trigger, threshold boundaries and the API contract. A set of [30] hand-written real-world messages, including typos, slang and Hinglish, is kept separate from the dataset and used as a robustness check.

6. Experiments and Results

6.1 Performance Targets

Component	Metric	Target	Achieved
Intent classifier	Macro F1	at least 0.90	[ ]
Intent classifier	Accuracy	at least 0.90	[ ]
FAQ retrieval	Top-1 accuracy	at least 0.75	[ ]
FAQ retrieval	Top-3 accuracy	at least 0.85	[ ]
Sentiment	Macro F1	at least 0.75	[ ]
Escalation	Recall on should-escalate cases	at least 0.90	[ ]

Component	Metric	Target	Achieved
Escalation	Precision	at least 0.70	[ ]
System	Median response latency	under 2 seconds	[ ]
System	Containment rate	at least 70%	[ ]

The targets were fixed before experiments began. Where a target was missed, Section 6.6 explains why.

6.2 Intent Classification Results

Model	Accuracy	Macro F1	Weighted F1	Train time
TF-IDF + Logistic Regression	[ ]	[ ]	[ ]	[ ]
MiniLM embeddings + Logistic Regression	[ ]	[ ]	[ ]	[ ]
DistilBERT (optional)	[ ]	[ ]	[ ]	[ ]

[Insert the confusion matrix of the best model and describe which intent pairs are confused most, and why.]

6.3 FAQ Retrieval Results

Metric	Value
Top-1 accuracy	[ ]
Top-3 accuracy	[ ]
Mean reciprocal rank	[ ]
Fallback rate at threshold [0.45]	[ ]

[Insert a short note on how the threshold was chosen, ideally with a precision-versus-coverage curve.]

6.4 Sentiment and Escalation Results

Model	Negative F1	Neutral F1	Positive F1	Macro F1
VADER	[ ]	[ ]	[ ]	[ ]
RoBERTa	[ ]	[ ]	[ ]	[ ]

Escalation method	Precision	Recall	F1
Rule-based engine	[ ]	[ ]	[ ]
Learned classifier (on `escalated` column)	[ ]	[ ]	[ ]

[State which was chosen and why. Recall matters more than precision here, because a missed escalation harms a student while an extra one only costs agent time.]

6.5 System-Level Results

Metric	Value
Median / 95th percentile latency	[ ] / [ ]
Containment rate (resolved without a human)	[ ]
Average user rating	[ ]
Small user survey (n = [ ]) satisfaction	[ ]

6.6 Error Analysis

Include at least five real failures from the test set in a table.

Student message	True intent	Predicted	Confidence	Why it failed
[ ]	[ ]	[ ]	[ ]	[e.g. ambiguous between two intents]
[ ]	[ ]	[ ]	[ ]	[e.g. Hinglish phrasing]
[ ]	[ ]	[ ]	[ ]	[e.g. typo or abbreviation]
[ ]	[ ]	[ ]	[ ]	[e.g. multiple questions in one message]

Student message	True intent	Predicted	Confidence	Why it failed
[ ]	[ ]	[ ]	[ ]	[e.g. query not covered by the FAQ]

Common failure modes to look for: near-synonym intents, very short messages, code-mixed text, queries that combine two topics, and out-of-scope questions that look in-scope. State which fixes were tried and which remain.

6.7 Robustness Check

On the [30] hand-written messages the best model scored [ ] accuracy versus [ ] on the test split. A large gap indicates that the paraphrased training data is cleaner than real student writing, which is discussed in Section 8.

7. Ethical Considerations and Privacy

The programme requires that the chatbot must not store student PII beyond the session, must include an opt-out, and must clearly indicate escalation to a human. These are implemented as follows.

Requirement	Implementation
No PII beyond the session	PII scrubbing before logging; conversation state held only in memory; session data discarded on close; tickets store masked text only
Opt-out	A visible opt-out control stops data capture and routes the student to a human
Clear escalation	A banner and message announce the handoff and state the reason, so the student always knows whether they are talking to a bot or a person
Transparency	The interface states at the start that this is an automated assistant
Fairness	Performance is checked across query types and across English and Hinglish messages
Safety for sensitive topics	Harassment, mental-health and grievance intents are never answered by the bot alone; they are escalated immediately with a message pointing to the appropriate human support

Requirement	Implementation
Grounded answers	The bot returns only knowledge-base text, never free-form generated policy

The signed Ethical Compliance Declaration accompanies this report.

8. Limitations

1. **Data realism.** The dataset contains paraphrased queries, which are cleaner than real student messages. Scores on the dataset may overstate real-world accuracy. The hand-written robustness set partly addresses this.
2. **Language coverage.** Hinglish and other code-mixed or regional-language input is only partly supported.
3. **Single-university scope.** The FAQ base reflects one institution; policies change and the base needs regular updates.
4. **Weak sentiment labels.** Ratings are a noisy proxy for sentiment, and VADER struggles with sarcasm.
5. **Single-turn focus.** The system handles each message with limited memory of earlier turns, so follow-up questions such as "and for the second semester?" can fail.
6. **Class imbalance.** Rare intents have fewer examples and lower recall.
7. **No live integration.** The bot cannot look up an individual student's fee status or results, only general policy.

9. Future Work

- Multilingual and Hinglish support with a multilingual encoder and code-mixed training data.
- Multi-turn dialogue management with slot filling and context carry-over.
- ERP and LMS integration for personalised answers, with authentication.
- A retrieval-augmented generation step that rephrases answers while staying strictly grounded in FAQ text.
- Active learning: route low-confidence queries to staff, then add the corrected examples to training data.
- Voice interface and WhatsApp or Telegram deployment.
- An agent dashboard with ticket assignment, response-time tracking and SLA alerts.

10. Conclusion

This project delivered a working helpdesk chatbot that combines intent classification, semantic FAQ retrieval, sentiment analysis and a transparent escalation policy. Measured against targets fixed in advance, the system achieved [summary of results: which targets were met and which were not]. The comparison between a classical TF-IDF baseline and an embedding-based model showed that [key insight]. Error analysis showed that most failures come from [key failure modes], which points to clear next steps. Most importantly, the design treats knowing when to hand over to a human as a first-class feature, backed by privacy-by-design choices that satisfy the programme's ethical requirements.

Code repository: [GitHub link] | **Demo video:** [link] | **Screenshots:** [Appendix D]

References

1. HCLTech Career Shaper Programme Guidelines, provided by the Programme Coordinator, Academic Year 2025-26.
2. Reimers, N. and Gurevych, I. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." EMNLP 2019.
3. Devlin, J. et al. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." NAACL 2019.
4. Sanh, V. et al. "DistilBERT, a distilled version of BERT." 2019.
5. Hutto, C. and Gilbert, E. "VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text." ICWSM 2014.
6. Barbieri, F. et al. "TweetEval: Unified Benchmark and Comparative Evaluation for Tweet Classification." 2020 (basis of the cardiffnlp sentiment models).
7. Pedregosa, F. et al. "Scikit-learn: Machine Learning in Python." JMLR 2011.
8. Johnson, J., Douze, M. and Jegou, H. "Billion-scale similarity search with GPUs." 2017 (FAISS).
9. [Mentor-validated reference 1]
10. [Mentor-validated reference 2]

Appendix A: Data Dictionary

Column	Type	Description	Example
session_id	string	Unique ID of a chat session	[ ]
user_query_type	categorical	Coarse category of the student's query	[ ]

Column	Type	Description	Example
intent	categorical	Labelled intent of the query (target variable)	[ ]
confidence_score	float, 0 to 1	Confidence reported by the original system	[ ]
escalated	Y/N	Whether the session was handed to a human agent	[ ]
user_rating	integer, 1 to 5	Student's satisfaction rating	[ ]
timestamp	datetime	When the message was sent	[ ]
paraphrased_query	text	Paraphrased student query (model input)	[ ]

Add any further columns present in your `dataset.csv` (for example `bot_response`) and export this table as `data_dictionary.csv`.

Appendix B: Sample Conversation

Turn	Speaker	Message	System decision
1	Student	[When is the last date to pay semester fees?]	Intent: [fee_payment], confidence [ ], retrieval [ ], answered
2	Bot	[FAQ answer]	Feedback buttons shown
3	Student	[This is the third time I am asking, nobody helps!]	Negative sentiment, repeated question: **escalated**
4	Bot	You are being connected to a human agent. Your ticket number is [ ].	Ticket created with masked transcript

Appendix C: How to Run

1. Create a virtual environment and run `pip install -r requirements.txt`.
2. Place `dataset.csv` and `faq.csv` in `data/`.
3. Train the models with `python src/intent.py --train`.
4. Start the API with `python api/app.py`.
5. Start the interface with `streamlit run ui/streamlit_app.py`.

Appendix D: Screenshots and Declaration

[Insert: chat interface, escalation banner, ticket queue, intent-distribution chart, confusion matrix.]

[Attach the signed Ethical Compliance Declaration.]